PHYSICAL UNCLONABLE FUNCTION WITH SEQUENTIAL STABILITY CLASSIFICATION FOR LIGHTWEIGHT IOT DEVICE AUTHENTICATION

A PROJECT REPORT

Submitted by

THARUN H (2116231901055)

UDAYKRISHNA NEELANOOTHALA (2116231901057)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled **"PHYSICAL UNCLONABLE FUNCTION WITH SEQUENTIAL STABILITY CLASSIFICATION FOR LIGHTWEIGHT IOT DEVICE AUTHENTICATION"** is the bonafide work of **"THARUN H. (2116231901055), UDAYKRISHNA NEELANOOTHALA (2116231901057)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

6/10/26

SIGNATURE

Dr. P. Tamilselvi, M.E, Ph.D.,

HEAD OF THE DEPARTMENT

Professor
Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

SIGNATURE

Mr. Benedict J.N.,

SUPERVISOR

Assistant Professor (SG)
Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 06/10/2026

Internal Examiner

6/10/26

External Examiner

ABSTRACT

Internet of Things (IoT) devices are increasingly deployed in environments where limited computational resources and physical exposure create significant security challenges. Conventional authentication approaches generally depend on stored cryptographic keys, certificates, or computationally expensive public-key mechanisms. Physical Unclonable Functions (PUFs) provide an alternative approach by deriving device-specific information from inherent physical variations introduced during semiconductor manufacturing. In existing system ESP32 with internal SRAM is used as the PUF source and it implements the client and server components on a personal computer. This project extends the underlying SRAM-PUF concept to an Arduino Uno based on the ATmega328P microcontroller. The proposed system extracts the SRAM startup pattern after repeated power cycles and applies Temporal Majority Voting (TMV) to reduce temporal instability. A Spatial Statistical Correlation (SSC) filtering stage is then applied to identify and reject highly correlated or structurally biased regions. The resulting stable SRAM cells are used to generate a device-specific fingerprint. A Python-based software platform is used for automated data acquisition, preprocessing, statistical analysis, fingerprint generation, and authentication. A software simulation has also been developed to validate the proposed processing pipeline before hardware deployment.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

THARUN H 2116231901055

UDAYKRISHNA N 2116231901057

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**viii**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	2
	1.4 SCOPE	3
	1.5 SECURITY REQUIREMENTS	4
	1.6 SDG ALIGNMENT	5
2	**LITERATURE REVIEW**	**6**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	6
	2.2 RESEARCH GAPS	10
	2.3 PROPOSED SYSTEM	11
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	11
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	12
	2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM	12
3	**SYSTEM DESIGN**	**13**
	3.1 SYSTEM ARCHITECTURE	13
	3.2 MODULE DESIGN	14
	3.3 DATA FLOW DIAGRAMS	15
	3.4 THREAT MODEL AND ATTACK SURFACE	17

CHAPTER NO.	TITLE	PAGE NO.
	3.5 SECURITY CONTROLS	18
	3.5.1 PHYSICAL DEVICE ZONE CONTROLS	18
	3.5.2 COMMUNICATION CHANNEL CONTROLS	19
	3.5.3 SERVER ZONE CONTROLS	20
	3.5.4 PROTOTYPE LIMITATIONS	21
4	**SYSTEM DESCRIPTION**	**22**
	4.1 IMPLEMENTATION OVERVIEW	22
	4.2 HARDWARE SETUP AND DATA COLLECTION	22
	4.2.1 TEST BENCH COMPONENTS	22
	4.2.2 SRAM CAPTURE FIRMWARE	23
	4.2.3 ACQUISITION PROCEDURE AND DATASET	23
	4.3 DATA PREPROCESSING AND ALGORITHMS	24
	4.3.1 TEMPORAL MAJORITY VOTING WITH SEQUENTIAL STABILITY CLASSIFICATION	24
	4.3.2 SPATIAL STATISTICAL CORRELATION FILTER	25
	4.3.3 FINGERPRINT GENERATION	25
	4.4 AUTHENTICATION PROTOTYPE	25
	4.5 WORK SCHEDULE FOR PHASE II	26
5	**CONCLUSION**	**27**
	5.1 CONCLUSION	27

CHAPTER NO.	TITLE	PAGE NO.
	REFERENCES	**28**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SDG ALIGNMENT	5
2.1	SUMMARY OF RELATED WORK	9
4.1	PLANNED HARDWARE COMPONENTS OF THE TEST BENCH	22
4.2	PLANNED EXPERIMENTS AND DATA COLLECTION	23

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	SYSTEM ARCHITECTURE	13
3.2	SEQUENCE DIAGRAM	15
3.3	DATA FLOW DIAGRAM	16
3.4	USE CASE DIAGRAM	17
3.5	THREAT MODEL DIAGRAM	18

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

The Internet of Things consists of large numbers of interconnected devices such as sensors, controllers, actuators and embedded systems. These devices frequently operate under restrictions on processing capability, memory, energy consumption and communication bandwidth. At the same time, many IoT devices are deployed in physically exposed environments and may become targets of spoofing, cloning, message replay, man-in-the-middle, side-channel and other attacks. The base paper identifies authentication and key exchange as important security requirements for resource-constrained IoT environments and discusses the limitations of traditional PKI and identity-based encryption approaches for such systems.

A Physical Unclonable Function provides a hardware-rooted approach to device identification. A PUF exploits physical variations introduced during manufacturing to generate a device-specific response. The response is not directly stored as a conventional secret but is derived from the physical characteristics of the device. The base paper describes a PUF using challenge-response behavior, in which a challenge is provided to the PUF and a corresponding response is generated. It also describes weak PUFs as suitable candidates for secret-key generation because of their relatively stable and reproducible responses.

SRAM-PUF is particularly suitable for low-cost embedded systems because it uses SRAM that is already present inside a microcontroller. When SRAM is powered on, individual cells tend to settle into preferred states because of small physical variations in the underlying circuitry. Repeated measurements can therefore reveal cells whose startup values are sufficiently stable to form a hardware fingerprint. The base paper uses the startup values of internal SRAM

to construct a 96-bit fingerprint and treats the addresses of the stable cells as PUF challenges.

The proposed work investigates the use of an ATmega328P-based Arduino Uno as a low-cost SRAM-PUF platform. The project introduces two processing stages, namely Temporal Majority Voting and Spatial Correlation/Statistical filtering, to improve the quality of the extracted SRAM response before using it for authentication.

1.2 PROBLEM STATEMENT

Conventional IoT authentication mechanisms frequently require secrets, certificates, or computationally expensive cryptographic operations to be stored or executed on resource-constrained devices. Such approaches increase storage, computational and communication overhead and may expose stored secrets to physical extraction or compromise.

Although SRAM-PUFs provide a hardware-derived identity, SRAM startup responses are not perfectly stable across repeated power cycles. Environmental variations and cell-level instability can cause individual bits to change between measurements. In addition, structurally correlated or locally biased regions of SRAM may reduce the quality of the generated fingerprint.

Therefore, the project addresses the problem of developing a low-cost and lightweight SRAM-PUF system on a single Arduino Uno that can extract a stable device-specific fingerprint by combining repeated SRAM startup measurements, Temporal Majority Voting and Spatial Correlation filtering, and subsequently demonstrate its use in a lightweight authentication mechanism.

1.3 OBJECTIVES

The objectives of the proposed project are:

- To investigate the power-up behavior of SRAM cells in the ATmega328P microcontroller and obtain raw SRAM startup responses through repeated power cycles.
- To develop an automated data acquisition system for collecting multiple SRAM startup measurements from a single Arduino Uno.
- To implement Temporal Majority Voting to reduce temporal noise and identify SRAM cells with stable startup behavior.
- To implement a spatial filtering method to identify locally correlated or structurally biased SRAM regions and improve the quality of the selected response bits.
- To generate a stable SRAM-PUF fingerprint from the selected SRAM cells.
- To evaluate the PUF using reliability, intra-chip Hamming distance, uniformity, entropy, usable-bit count and other suitable statistical metrics.
- To develop a Python-based enrollment and authentication demonstration using the generated PUF response.
- To compare the raw SRAM response with TMV-filtered and TMV+SSC-filtered responses.

1.4 SCOPE

- The project focuses on the implementation of an SRAM-based Physical Unclonable Function using the ATmega328P present in an Arduino Uno. The initial implementation uses a single physical Arduino Uno and therefore concentrates primarily on temporal stability, reliable cell selection, data acquisition, filtering, environmental robustness and authentication workflow.
- The scope includes hardware-controlled power cycling, early SRAM startup capture, PC-based data logging, TMV processing, spatial statistical filtering, fingerprint generation and lightweight

authentication. The PC acts as the processing platform for data collection, analysis and server/client emulation.

- The software simulation developed during the preliminary phase is used to validate the algorithmic pipeline before hardware experimentation. The current simulation models different virtual SRAM characteristics and demonstrates the effect of TMV and spatial filtering.
- Because only one physical Arduino is used in the current hardware plan, experimental **inter-chip uniqueness cannot yet be established from hardware measurements**. Inter-chip behavior may be studied initially through simulation and can later be experimentally validated using multiple ATmega328P devices.

1.5 SECURITY REQUIREMENTS

The proposed system is designed around the following security requirements:

- **Device-specific identity:** The authentication information should be derived from physical properties of the microcontroller rather than from a permanently stored conventional secret.
- **Response stability:** Repeated power-up measurements from the same device should produce sufficiently similar responses for reliable authentication.
- **Uniqueness:** Different physical devices should ideally exhibit sufficiently different responses. This property will require additional physical devices for experimental validation.
- **Resistance to replay:** The authentication mechanism should use fresh challenges, nonces or other session-dependent values rather than transmitting a fixed response repeatedly.
- **Response confidentiality:** Raw SRAM startup values should not be unnecessarily exposed over the communication channel.

- **Lightweight processing:** Operations suitable for resource-constrained systems should be preferred. The base paper specifically uses PUF together with XOR and hash operations and avoids computationally expensive ECC-based processing in its proposed protocol.
- **Physical security:** The system should consider attacks involving direct physical access to the device, observation of communication, response collection and attempts to reproduce the PUF behavior.

1.6 SDG ALIGNMENT

Table 1.1 SDG ALIGNMENT

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 9 - Industry, Innovation and Infrastructure	9.1 - Develop quality, reliable, sustainable and resilient infrastructure	Gives IoT devices used in infrastructure a hardware-rooted identity, which reduces the risk of device cloning and spoofing
2	SDG 9 - Industry, Innovation and Infrastructure	9.5 - Enhance scientific research and upgrade technological capabilities	Builds a reproducible research methodology (TMV + SSC) for characterising SRAM-PUFS on low-cost microcontrollers.
3	SDG 9- Industry, Innovation and Infrastructure	9.c-Increase access to ICT and provide affordable access	Uses SRAM that already exists inside a commodity ATmega328P, so secure identification is available without costly secure elements
4	SDG 12- Responsible Consumption and Production	12.5-Substantially reduce waste generation	Reuses an existing on-chip resource as the security primitive instead of adding dedicated security hardware, which lowers the component count per device.

CHAPTER 2
LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Conventional IoT device authentication relies on cryptographic keys or certificates stored in the device's non-volatile memory. The security of these schemes depends on keeping the key secret and protected against physical access, which requires a secure storage perimeter that low-cost microcontrollers lack [3]. An attacker who captures an unattended device can read its flash or EEPROM, copy its firmware and clone it onto identical hardware [3]. Public-key infrastructure adds certificate generation, storage, verification and revocation costs. Identity-based encryption avoids certificates but relies on pairing-based operations and a central key generator. Both are therefore poorly suited to resource-constrained IoT nodes [10].

Physical Unclonable Functions (PUFs) offer an alternative: a device-specific secret is derived from uncontrollable manufacturing variation instead of being stored. Suh and Devadas [18] showed how silicon PUFs can support both device authentication and secret-key generation. Maes and Verbauwhede [12] classify PUFs into strong PUFs, with an exponentially large challenge–response space, and weak PUFs, with a limited response set suited to key generation. They identify reliability, uniqueness and unpredictability as the core quality properties.

Memory-based PUFs exploit the power-up state of SRAM. Guajardo et al. [9] and Holcomb et al. [11] independently showed that uninitialised SRAM settles into a device-specific pattern at power-up. Each six-transistor cell is a pair of cross-coupled inverters, and small threshold-voltage mismatches push it towards a preferred state. Holcomb et al. also showed that repeated power-ups separate strongly skewed, stable cells from near-symmetric, noisy cells [11].

Schrijen and van der Leest [17] compared SRAMs from several technology nodes and found large differences in stability and entropy between memory types. SRAM-PUF quality must therefore be measured on each specific device rather than assumed.

Several studies target Arduino-class microcontrollers directly. Elmestekawi et al. [7] proposed a key generator combining the chip signature and SRAM-PUF of the ATmega328P. Aung et al. [2] evaluated SRAM-PUF characteristics and proposed a method for generating stable bits for IoT security. Farha et al. [8] proposed an SRAM-PUF entity authentication scheme for constrained IoT devices using BCH-based error correction. Amar et al. [1] later showed that the XOR-based response reconstruction in that scheme can be exploited.

The selected base paper, Barbareschi et al. [3], is the most complete characterisation of the ATmega328P SRAM-PUF to date. The authors dumped SRAM content from 113 Arduino Uno boards, taking 30 dumps per device at 5 V and about 25 °C, with a 15-second power-off between dumps to avoid retention effects. They reported a mean instability of about 0.02, mean uniqueness of 0.49, and uniformity and bit-aliasing of about 0.55. Varying the supply from 2.7 V to 5.5 V left instability almost constant, while testing from −20 °C to 85 °C raised average instability to about 0.069 at the coldest point. To derive a 128-bit key, they selected a Reed–Muller RM(1,7) fuzzy extractor needing 2432 source bits, integrated into a 20 KB secure bootloader with an average key-generation time of about 2080 ms. The paper also proposes HKDF-based device–server and device–device authentication schemes [3].

Raw SRAM-PUF responses are noisy, so key-generation systems commonly use helper-data algorithms. The fuzzy extractor of Dodis et al. [6] combines an error-correcting code for information reconciliation with a hash for privacy amplification. Delvaux et al. [5] surveyed helper-data algorithms for PUF-based key generation and analysed their security pitfalls. In all such schemes, the

required ECC size and failure rate depend directly on the bit error rate of the input. The base paper [3] shows the resulting cost on an 8-bit microcontroller.

A complementary approach reduces the error rate before error correction by selecting only reliable cells. Holcomb et al. [11] showed that repeated power-up observations identify consistently stable cells. Mathew et al. [15] combined temporal majority voting (TMV) with masking of unstable bits to achieve highly stable key generation in a 22 nm hybrid PUF. Xiao et al. [21] proposed a bit-selection algorithm for SRAM-PUFs suited to high-volume production. Aung et al. [2] applied stable-bit generation to IoT-oriented SRAM-PUFs. Such pre-selection lowers the residual bit error rate and can reduce the ECC overhead required downstream.

Most SRAM-PUF entropy estimates assume that response bits are independent. Wilde, Gammel and Pehl [20] showed that PUF responses can contain spatial correlation linked to the physical layout. Such correlation reduces effective entropy below what per-bit statistics suggest, and it can be detected with dedicated spatial analysis. The base paper itself computes min-entropy under an explicit bit-independence assumption [3].

To compare PUF implementations, standard metrics are required. Maiti et al. [13] proposed a systematic method for evaluating PUFs using uniformity, reliability, uniqueness, bit-aliasing and related measures, which this project adopts.

At the protocol level, Mall et al. [14] survey PUF-based authentication and key-agreement protocols across IoT, WSN and smart-grid settings. Gupta and Varshney [10] proposed a lightweight authentication and key-exchange protocol using only PUF, bitwise XOR and hash operations. Their prototype runs on an ESP32 and builds a 96-bit fingerprint from stable SRAM cells, whose addresses serve as challenges. On the security side, Rührmair et al. [16] showed that many strong PUFs can be modelled with machine learning from observed challenge–

response pairs. This motivates keeping raw PUF responses off the communication channel.

Table 2.1 SUMMARY OF RELATED WORK

Ref	Platform	Approach	Limitation relevant to this work
[11]	Commodity SRAM	Power-up state as fingerprint and TRNG	Not targeted at 8-bit MCU authentication
[7]	ATmega328P	Chip signature + SRAM-PUF key generator	Small device set; no spatial analysis
[2]	Microcontroller SRAM	SRAM-PUF evaluation and stable-bit generation	No spatial correlation analysis or staged evaluation
[8]	IoT MCU incl. Arduino	SRAM-PUF entity authentication with BCH	XOR reconstruction later shown exploitable [1]
[3]	113 × ATmega328P	Characterisation + RM(1,7) fuzzy extractor + secure bootloader	No pre-ECC cell selection; bit independence assumed
[15]	22 nm ASIC	TMV + unstable-bit masking	Custom silicon, not COTS MCU
[20]	Various PUFs	Spatial correlation analysis	Not applied to ATmega328P SRAM
[10]	ESP32	PUF + XOR + hash authentication, 96-bit fingerprint	32-bit platform; protocol focus, not PUF quality

2.2 RESEARCH GAPS

The reviewed literature establishes that the ATmega328P SRAM produces a usable PUF response [3], [7], and that lightweight PUF-based authentication is feasible on IoT devices [10], [8]. However, the following gaps remain in how the response is processed before it is used:

- **No pre-ECC cell selection on the ATmega328P.** The base paper passes raw SRAM key material to a fuzzy extractor sized for the worst-case instability. The result is a large source-bit requirement and roughly two seconds of key generation on the device [3]. Reliability-based selection such as TMV and unstable-bit masking [15], [21] can lower the residual error rate, but its effect on the ATmega328P has not been quantified in [3], [7] or [8].
- **Bit independence is assumed rather than tested.** The base paper's min-entropy estimate treats response bits as independent [3]. Spatial correlation, if present, would make that estimate optimistic [20]. None of the reviewed Arduino SRAM-PUF studies tests for neighbourhood correlation.
- **Bias is reported but not filtered.** Per-device bit-aliasing in the base paper ranges from about 0.37 to 0.72 around an average near 0.55. This indicates biased cells or regions, yet no reviewed Arduino work proposes a statistical filter to remove them before fingerprint generation.
- **No stage-wise attribution.** Existing works report final quality metrics or key failure rates. The individual contribution of each processing

stage (raw → temporal filtering → spatial filtering) is not separated. This project proposes to measure each stage independently.

- **Authentication on 8-bit responses needs care.** Farha et al. [8] had to shorten their fingerprint and weaken error correction on Arduino, and their XOR reconstruction was later shown to be exploitable [1]. The XOR/hash protocol of [10] was demonstrated on a 32-bit ESP32. Applying such a protocol to responses from an 8-bit ATmega328P, without exposing raw responses on the channel, remains to be demonstrated.
- **Characterisation and authentication are often coupled.** This project separates the PUF characterisation problem from the authentication problem, so that reliability and filtering improvements can be quantified before authentication is integrated.

To the best of our knowledge, combining TMV-based cell selection with spatial statistical filtering, and evaluating each stage separately on the ATmega328P, has not been reported. This combination is the focus of the proposed work.

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system uses the SRAM of a single ATmega328P microcontroller as the physical source of the device fingerprint. SRAM startup values are captured over multiple controlled power-up cycles and stored on the PC for analysis. The raw measurements are first processed with Temporal Majority Voting: for every SRAM bit the values observed over the power cycles are compared, the majority state is selected, and cells with insufficient stability are removed from the candidate set.

A spatial statistical/correlation filter is then applied to the TMV output. The purpose of the spatial stage is to detect regions in which neighboring cells exhibit unusually similar or biased behavior and to remove such cells from the final response. The remaining stable cells are used to form the fingerprint.
Following the base paper, the addresses of the selected cells serve as the challenge and their values as the response, with TMV and SSC inserted as additional preprocessing stages. The final fingerprint is used by the Python-based authentication system, in which the PC emulates the Client and Server while the Arduino acts as the IoT device.

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

The proposed system realises the objectives of Section 1.3 as one pipeline: automated extraction of SRAM startup values, TMV to reduce temporal instability, SSC to remove biased or correlated cells, and a stable, statistically evaluated response used for lightweight authentication.

2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM

The proposed architecture offers several advantages. It is low in cost, because the PUF source is already present inside the ATmega328P and no dedicated PUF hardware is needed. Its identity is hardware-rooted, since the response comes from physical SRAM behaviour rather than from a fixed identifier stored in non-volatile memory. TMV reduces temporal noise by using repeated observations to identify stable startup states, and SSC adds spatial quality control by rejecting locally biased or correlated regions. The security layer remains lightweight, relying on hash and XOR operations in line with the base paper. Python automates the whole workflow, and the architecture extends naturally to multiple devices and environmental conditions.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

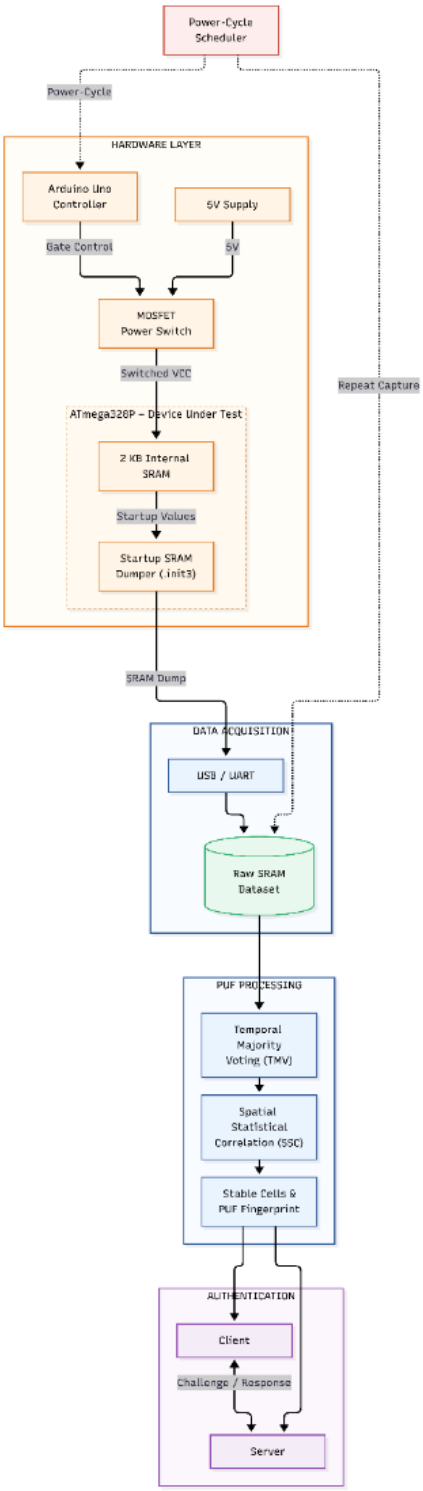


Figure 3.1 System architecture

3.2 MODULE DESIGN

MODULE 1 - SRAM STARTUP CAPTURE

This module is responsible for obtaining the initial SRAM state after power-up. The capture must occur sufficiently early in the microcontroller startup process so that normal program initialization does not overwrite the SRAM values of interest.

MODULE 2 - AUTOMATED POWER CYCLING

The target Arduino is repeatedly powered off and powered on using an external MOSFET or relay power-control circuit. This creates independent startup events required for repeated SRAM measurements.

MODULE 3 - DATA ACQUISITION

Captured SRAM values are transmitted to the PC over USB/UART. Python receives the measurements, validates the packet structure and stores the raw data without modification.

MODULE 4 - TEMPORAL MAJORITY VOTING

This module compares each SRAM cell across repeated power cycles and determines its dominant startup state. A confidence or reliability value is also calculated for every cell.

MODULE 5 - SPATIAL STATISTICAL CORRELATION

This module 14nalyses the spatial neighborhood of candidate cells after TMV. Cells belonging to highly uniform or correlated local regions can be rejected according to the selected spatial criterion.

MODULE 6 - FINGERPRINT GENERATION

The valid SRAM cells are extracted and converted into a fixed-length binary response. Their addresses constitute the challenge information used by the PUF system.

MODULE 7 - AUTHENTICATION

The extracted PUF response is incorporated into a lightweight authentication mechanism. The Python Client and Server represent the communication entities, while the Arduino represents the IoT device.

MODULE 8 - PERFORMANCE ANALYSIS

The final module calculates Hamming distance, reliability, entropy, uniformity, usable-bit count and other experimental measurements and generates plots for comparison.

3.3 DATA FLOW DIAGRAMS

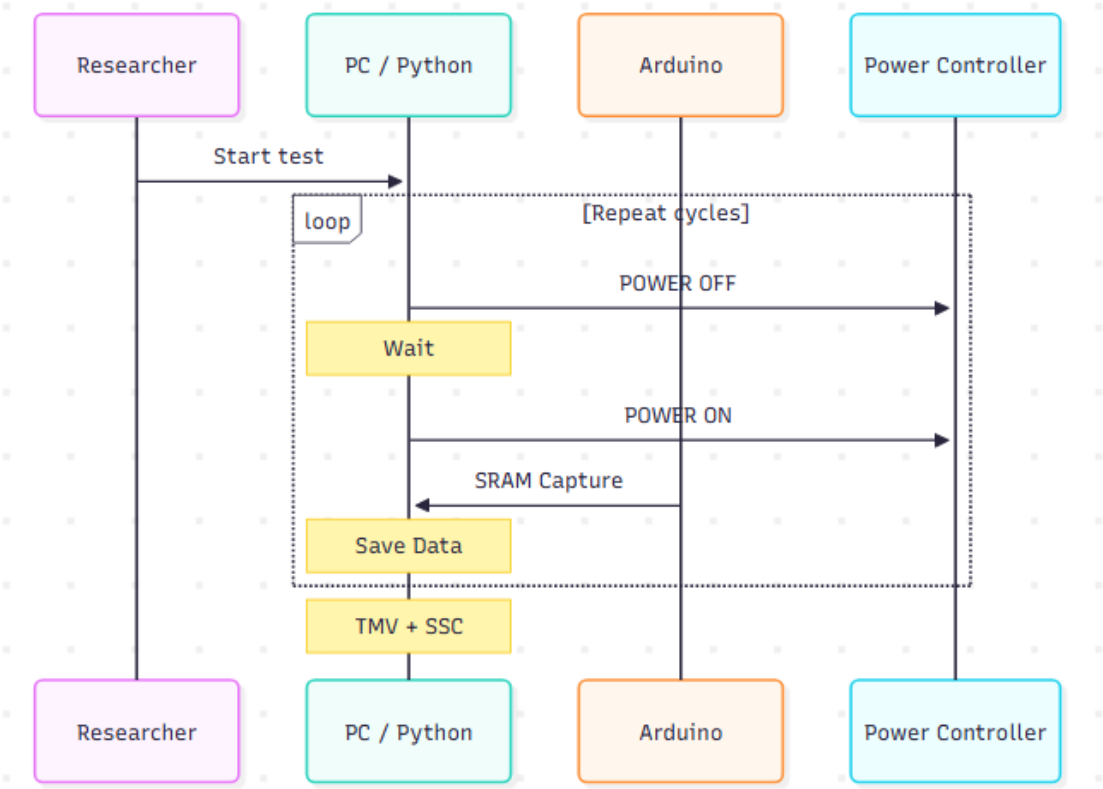


Figure 3.2 Sequence diagram

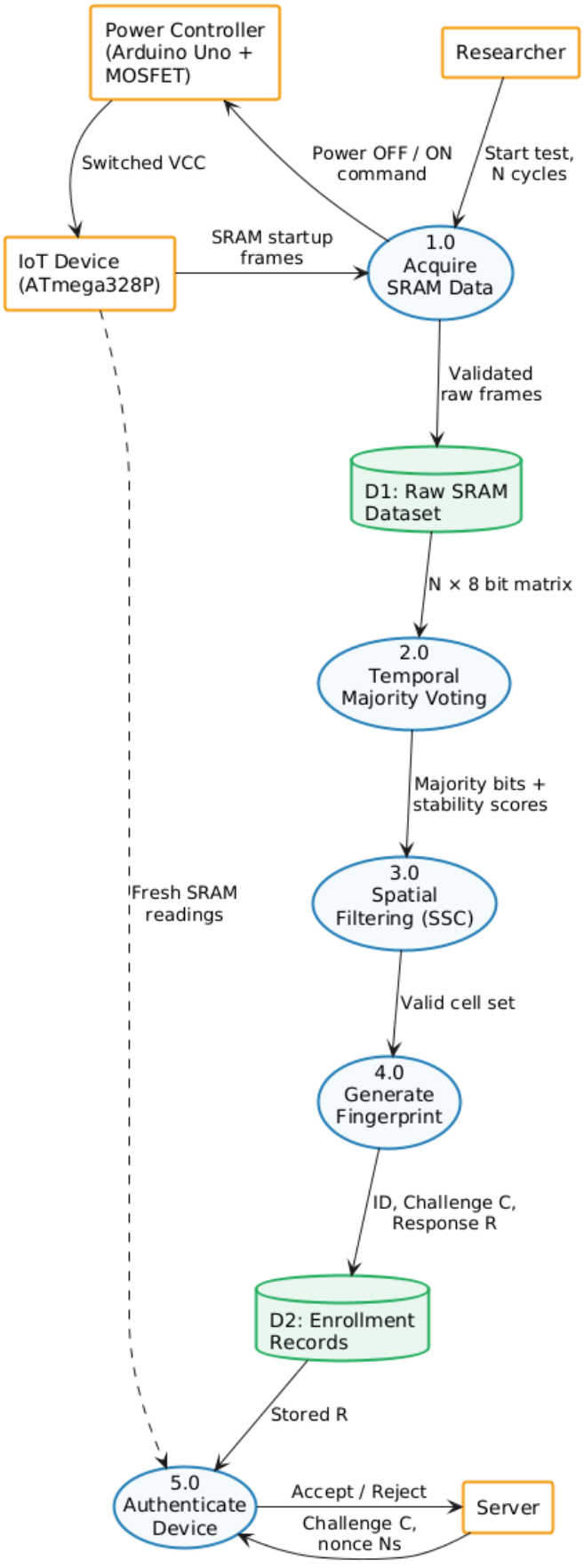


Figure 3.3 Data flow diagram

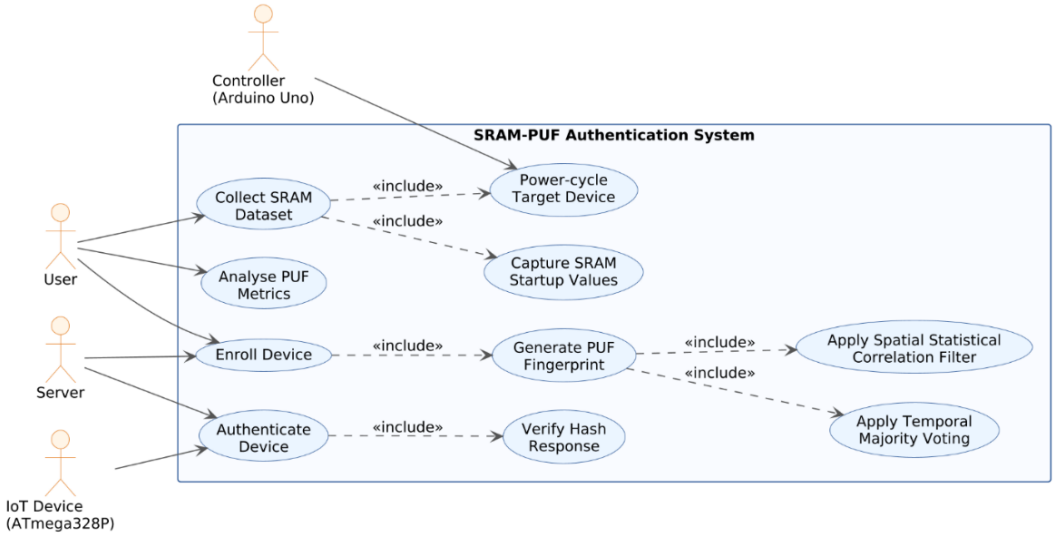


Figure 3.4 Use case diagram

3.4 THREAT MODEL AND ATTACK SURFACE

The proposed system considers an attacker who can observe or manipulate the communication between the device and the PC-based authentication components, and who may replay or modify messages, impersonate the Arduino, or collect and analyse observed responses. Raw SRAM responses are therefore treated as sensitive, and only hash-derived values are used for authentication. The major attack surfaces in the prototype are:

- **SRAM capture interface:** An attacker with physical access may attempt to directly obtain startup SRAM values.
- **USB/UART interface:** Raw or insufficiently protected responses may be observed.
- **Authentication messages:** An attacker may attempt replay or modification.
- **Challenge-response relationship:** Excessive exposure of raw PUF responses can provide information useful for modeling.
- **Power supply:** Manipulation of voltage and startup conditions may affect PUF reliability.

- **Firmware:** Modification of the firmware may allow an attacker to replace the intended PUF operation.

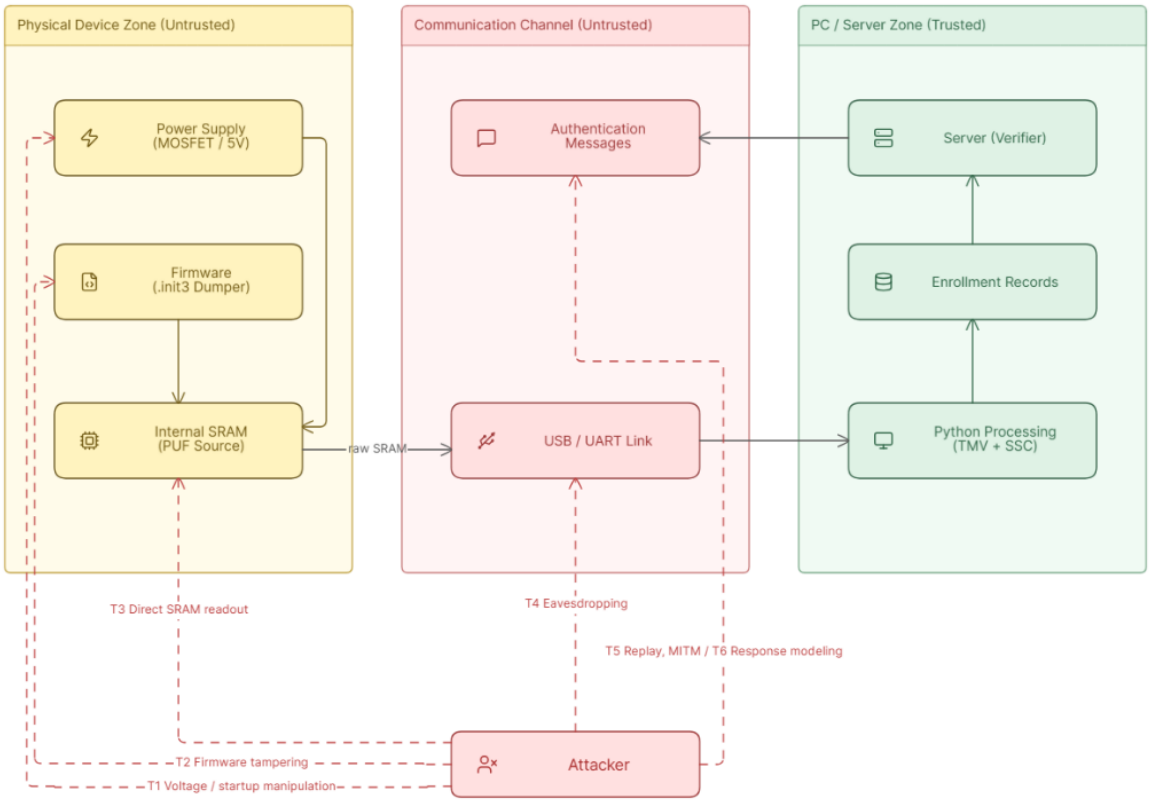


Figure 3.5 Threat model diagram

3.5 SECURITY CONTROLS

The six threats identified in Section 3.4 span three trust zones: the Physical Device Zone, the Communication Channel and the PC/Server Zone. The controls below follow the lightweight principles of the base paper, namely PUF-derived responses, fresh random values, XOR operations and hash functions, without computationally expensive public-key cryptography. Because the project is a research prototype, some controls belong to the current design and others are proposed for later phases.

3.5.1 PHYSICAL DEVICE ZONE CONTROLS

C1 - Hardware-Rooted Identity (T3)

The device identity comes from the physical startup behaviour of the ATmega328P's internal SRAM. No cryptographic key is stored in flash or EEPROM. An attacker who reads the device's non-volatile memory therefore finds no stored credential, and the identity cannot be copied into another chip, because it depends on manufacturing variation.

C2 - Controlled Hardware Power Cycling (T1)

SRAM startup values are captured only after a full power cycle controlled by the MOSFET, with a fixed discharge interval between cycles. A software reset is never used, because SRAM keeps its previous contents across a reset. This ensures every measurement is a genuine startup state and prevents data remanence from corrupting the dataset.

C3 - Early SRAM Capture (T2)

The capture routine runs in the .init3 section, before the C runtime initialises .data and .bss. The startup values are read before any program code can change them. The stack region at the top of SRAM is excluded from analysis, because it is written during startup.

C4 - Firmware Protection (T2) - Proposed

Programming the ATmega328P's lock bits after deployment can block external read-back and modification of the flash. This makes it harder for an attacker to replace the capture routine with malicious firmware.

3.5.2 COMMUNICATION CHANNEL CONTROLS

C5 - Fresh Nonces (T5)

Every authentication session uses a new server nonce N_s and a new device nonce N_c. Each response is bound to its session, so a captured message cannot be replayed later.

C6 - Hash-Protected Responses (T4, T5, T6)

The raw PUF response R is never sent during authentication. The device side sends only $h = H(ID \parallel R' \parallel N_s \parallel N_c)$. An eavesdropper therefore cannot recover R from the channel, and any change to the message makes the hash check fail.

C7 - XOR Masking (T4, T5)

As in the base paper, protocol values are masked using XOR with PUF-derived or random values before transmission. This keeps the protocol lightweight enough for resource-constrained devices.

C8 - Frame Validation (T4, T5)

Each SRAM frame sent over UART carries a header, length field and checksum. Frames that are corrupted, incomplete or injected are rejected before storage.

3.5.3 SERVER ZONE CONTROLS

C9 -Trusted Enrollment

Enrollment happens once, in a controlled environment, before the device is deployed. The Server stores the enrollment record (ID, challenge C, response R) in a local database that is never exposed to the communication channel.

C10 - Temporal Majority Voting (T1)

Only SRAM cells that stay stable across all enrollment power cycles are selected. This reduces the effect of voltage and environmental changes, so authentication works reliably even under minor startup manipulation.

C11 - Spatial Statistical Correlation Filtering (T6)

SRAM windows that show bias or correlation with their neighbours are removed from the fingerprint. Predictable cell patterns would make the response easier to model, so the remaining bits are both stable and hard to predict.

C12 - Session Key Derivation (T5)

After successful verification, both parties derive a session key $K = H(R \| N_s \| N_c)$. Further communication is then bound to the device's physical identity, which prevents a man-in-the-middle from continuing the session.

C13 - Local Data Storage

All raw SRAM data and enrollment records are stored in a local SQLite database rather than cloud storage. This limits exposure of sensitive PUF data and fits the project's hardware-security premise. The raw dataset is kept only for research analysis and is never used directly as a credential.

3.5.4 PROTOTYPE LIMITATIONS

In the current prototype, the PC processes the raw SRAM dump, so raw startup values cross the USB/UART link during enrollment and analysis. As a result, T3 and T4 are only partly addressed during data collection. A full mitigation would move TMV reconstruction and hashing onto the device, so that only hashed values ever leave it. This is left as future work.

CHAPTER 4
SYSTEM DESCRIPTION

4.1 IMPLEMENTATION OVERVIEW

The design of Chapter 3 will be implemented in Phase II in four stages: a test bench with hardware power cycling; capture firmware and acquisition software; the sequential stability classifier, the spatial statistical correlation (SSC) filter and fingerprint generation; and the authentication protocol with its evaluation. No results are reported here; the algorithms, parameters and analysis are fixed in advance so that the method cannot be tuned to the observed data. The prototype scoped in Chapter 1 used a single Arduino Uno. Phase II extends it to five ATmega328P-PU microcontrollers as devices under test (DUTs), with one Arduino Uno R3 as controller, so that inter-chip uniqueness can be measured on hardware.

4.2 HARDWARE SETUP AND DATA COLLECTION

4.2.1 TEST BENCH COMPONENTS

The planned components are listed in Table 4.1.

Table 4.1 PLANNED HARDWARE COMPONENTS OF THE TEST BENCH

Component	Qty	Purpose
ATmega328P-PU with 16 MHz crystal	5	Devices under test (no bootloader)
Arduino Uno R3	1	Power sequencing and ISP programming
P-MOSFET switch and bleeder resistor	5	DUT power switching and discharge
CD4051 mux and CP2102 USB–UART	1	DUT output to PC at 115200 baud
Regulated supply and host PC	1 each	4.5–5.5 V supply; Python and SQLite

The DUTs are programmed without a bootloader because, on a standard Arduino Uno, Optiboot runs first and overwrites part of the SRAM before its startup values can be read. High-side switching keeps a common ground. During the off period the controller pins connected to a DUT are set as high-impedance inputs, so that the DUT cannot be partly powered through its I/O protection diodes, and the bleeder resistor discharges the decoupling capacitor so that the cells lose their previous contents.

4.2.2 SRAM CAPTURE FIRMWARE

The firmware will be written in C for avr-gcc and flashed with avrdude. The dump routine is placed in the .init3 section, which runs after the stack pointer is set but before the C runtime initialises .data and .bss in .init4. Declared naked and free of function calls, it never writes to the stack. It streams all 2048 bytes of SRAM (0x0100 to 0x08FF, 16,384 cells) in a frame with a start marker, DUT identifier, cycle counter, length and CRC-16 checksum (control C8). A guard region at the top of SRAM, provisionally 64 bytes, is excluded from analysis.

4.2.3 ACQUISITION PROCEDURE AND DATASET

Table 4.2 PLANNED EXPERIMENTS AND DATA COLLECTION

ID	Experiment	Conditions	Power-ups per DUT	Purpose
E1	Retention check	T_{off} = 1, 5, 10, 15, 30 s	50 per setting	Select a safe off interval
E2	Nominal enrollment	5.0 V, room temperature	500	Baseline for all three stages
E3	Supply voltage	4.5 V and 5.5 V	200 per setting	Voltage robustness
E4	Temperature	Cooled and heated enclosure	200 per setting	Thermal robustness
E5	Convergence	Replay of E2 data	–	Power-ups needed per cell
E6	Authentication	Genuine and impostor attempts	200 genuine per DUT	FRR, FAR and latency

The Python program instructs the controller to switch a DUT off for an interval Toff, switch it on and route its output to the PC. Each frame is validated and stored unmodified in SQLite with the DUT identifier, cycle number, supply voltage, temperature and timestamp (control C13). Barbareschi et al. [3] used a 15-second off interval, which E1 will verify for this circuit. A power cycle then takes about 16 s, so the 500 nominal power-ups of E2 need about 2.2 hours per DUT and produce about 5.2 MB in total. Six experiments are planned, as summarised in Table 4.2. E1 is the retention check, sweeping the off interval from 1 s to 30 s. E2 is the nominal enrollment of 500 power-ups per DUT at 5.0 V and room temperature, which provides the baseline for all three processing stages. E3 and E4 test robustness with 200 power-ups per setting at supply voltages of 4.5 V and 5.5 V and in cooled and heated enclosures. E5 replays the E2 data to measure how many power-ups each cell needs before it is classified, and E6 runs genuine and impostor authentication attempts to measure the false rejection rate, false acceptance rate and latency.

4.3 DATA PREPROCESSING AND ALGORITHMS

4.3.1 TEMPORAL MAJORITY VOTING WITH SEQUENTIAL STABILITY CLASSIFICATION

Let $x_i^{(t)} \in \{0, 1\}$ be the startup value of cell i at power-up t. After n power-ups, the probability that the cell starts at logic 1 is estimated as the fraction of those power-ups in which it started at 1. Its TMV value r_i is 1 if this estimate is at least 0.5 and 0 otherwise, and its flip probability is the probability that a power-up differs from r_i.

Instead of a fixed number of power-ups for every cell, the proposed classifier is sequential. After each power-up it computes, for every undecided cell with k_i flips, the exact Clopper–Pearson interval $[L_i(n), U_i(n)]$ for the flip probability at confidence $1 - \delta$ [4]. A cell is stable when $U_i(n) \leq \theta$, unstable when $L_i(n) > \theta$,

and undecided otherwise. For a cell that never flips, the upper bound falls below θ after a minimum number of power-ups, n_{min}, which depends only on θ and δ. With the provisional values $\theta = \delta = 0.05$, this minimum is $n_{min} = 59$ power-ups. Strongly skewed cells are therefore decided early, while noisy cells are rejected once flips accumulate. Enrollment stops when the undecided fraction falls below u_{max} or n reaches N_{max} (provisionally 500).

4.3.2 SPATIAL STATISTICAL CORRELATION FILTER

The SSC filter operates on the TMV values of the stable cells in address order. The SRAM is divided into windows W of 16 bytes (128 cells). The bias B_W of a window is the fraction of its cells whose TMV value is 1, and the window is rejected when B_W differs from 0.5 by more than a two-sided binomial test allows for an unbiased window.

Neighbourhood correlation is measured with the autocorrelation of the TMV values inside each window at lag 1 (adjacent bits in a byte) and lag 8 (the same bit in adjacent bytes). A window is also rejected when this correlation is significant at either lag, with the significance level Bonferroni-corrected across the 128 windows. The test is motivated by Wilde et al. [20].

4.3.3 FINGERPRINT GENERATION

Stable cells in accepted windows form the candidate set. The challenge C is an ordered list of L = 128 addresses drawn from this set, with no two cells from the same byte, and the response R holds their TMV values. The size of the candidate set after each stage (the usable-bit count) will be reported for every DUT.

4.4 AUTHENTICATION PROTOTYPE

The prototype follows the controls of Section 3.5. Enrollment stores (ID, C, R) in the server database once per DUT. In each session the server sends C

and a fresh nonce N_s; the device side reconstructs R′ by majority voting over m fresh power-ups (provisionally m = 5), chooses a nonce Nc and returns h = H(ID ‖ R′ ‖ N_s ‖ Nc). The server accepts if h matches the value computed with the enrolled R, and both sides derive K = H(R ‖ N_s ‖ Nc). Because the hash comparison requires R′ to equal R exactly, majority voting over several fresh power-ups is what makes the response reproducible at authentication time. If the measured success rate proves insufficient, a fuzzy extractor [6] can be added without changing the rest of the pipeline.

4.5 WORK SCHEDULE FOR PHASE II

Phase II is planned over twelve weeks. Weeks 1 to 3 cover assembly of the test bench, the capture firmware, the acquisition software and the retention check (E1). Weeks 4 to 6 cover nominal data collection (E2) and implementation of the sequential classifier and the SSC filter. Weeks 7 to 9 cover the voltage, temperature and convergence experiments (E3 to E5), together with the authentication prototype and its trials (E6). Weeks 10 to 12 are reserved for completing the analysis and preparing the journal manuscript and the final thesis.

CHAPTER 5
CONCLUSION

5.1 CONCLUSION

This report has presented the Phase I work on a low-cost SRAM-PUF for authenticating ATmega328P-based IoT devices. The literature review showed that the ATmega328P SRAM yields a usable fingerprint, but that existing Arduino studies pass the raw response to error correction without selecting reliable cells, assume bit independence when estimating entropy, and do not separate the effect of each processing stage. In this project we designed a pipeline that combines hardware power cycling and early SRAM capture with temporal majority voting, sequential stability classification and spatial statistical correlation filtering, followed by hash-based lightweight authentication, with thirteen security controls mapped to a three-zone threat model.

REFERENCES

1. Amar M., Kama A., Wang K. and Oren Y. (2022), 'Comment on "SRAM-PUF-based entities authentication scheme for resource-constrained IoT devices"', Cryptology ePrint Archive.
2. Aung P.P., Mashiko K., Ismail N.B. and Yee O.C. (2020), 'Evaluation of SRAM PUF characteristics and generation of stable bits for IoT security', Emerging Trends in Intelligent Computing and Informatics, Springer, pp.441-450.
3. Barbareschi M., Cirillo F., Esposito C. and Mazzocca N. (2025), 'Engineering SRAM-PUF on Arduino microcontroller', Journal of Systems Architecture, Vol. 167, Art. No. 103466.
4. Clopper C.J. and Pearson E.S. (1934), 'The use of confidence or fiducial limits illustrated in the case of the binomial', Biometrika, Vol. 26, No. 4, pp.404-413.
5. Delvaux J., Gu D., Schellekens D. and Verbauwhede I. (2015), 'Helper data algorithms for PUF-based key generation: Overview and analysis', IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, Vol. 34, No. 6, pp.889-902.
6. Dodis Y., Reyzin L. and Smith A. (2004), 'Fuzzy extractors: How to generate strong keys from biometrics and other noisy data', Advances in Cryptology – EUROCRYPT 2004, LNCS Vol. 3027, Springer, pp.523-540.
7. Elmestekawi A., Tammam A. and Issa H. (2018), 'Unclonable key generator based on chip signature and SRAM-PUF of ATmega328P chip', Proc. 28th Int. Conf. on Computer Theory and Applications (ICCTA), pp.24-29.
8. Farha F., Ning H., Ali K., Chen L. and Nugent C. (2021), 'SRAM-PUF-based entities authentication scheme for resource-constrained IoT devices', IEEE Internet of Things Journal, Vol. 8, No. 7, pp.5904-5913.
9. Guajardo J., Kumar S.S., Schrijen G.-J. and Tuyls P. (2007), 'FPGA intrinsic PUFs and their use for IP protection', Cryptographic Hardware and Embedded Systems – CHES 2007, LNCS Vol. 4727, Springer, pp.63-80.
10. Gupta C. and Varshney G. (2023), 'A lightweight and secure PUF-based authentication and key-exchange protocol for IoT devices', arXiv preprint arXiv:2311.04078.

11. Holcomb D.E., Burleson W.P. and Fu K. (2009), ‘Power-up SRAM state as an identifying fingerprint and source of true random numbers’, IEEE Transactions on Computers, Vol. 58, No. 9, pp.1198-1210.

12. Maes R. and Verbauwhede I. (2010), ‘Physically unclonable functions: A study on the state of the art and future research directions’, in Sadeghi A.-R. and Naccache D. (Eds.), Towards Hardware-Intrinsic Security, Springer, Berlin, pp.3-37.

13. Maiti A., Gunreddy V. and Schaumont P. (2013), ‘A systematic method to evaluate and compare the performance of physical unclonable functions’, Embedded Systems Design with FPGAs, Springer, pp.245-267.

14. Mall P., Amin R., Das A.K., Leung M.T. and Choo K.-K.R. (2022), ‘PUF-based authentication and key agreement protocols for IoT, WSNs, and smart grids: A comprehensive survey’, IEEE Internet of Things Journal, Vol. 9, No. 11, pp.8205-8228.

15. Mathew S.K. et al. (2014), ‘A 0.19pJ/b PVT-variation-tolerant hybrid physically unclonable function circuit for 100% stable secure key generation in 22nm CMOS’, IEEE Int. Solid-State Circuits Conf. (ISSCC) Digest of Technical Papers, pp.278-279.

16. Rührmair U., Sehnke F., Sölter J., Dror G., Devadas S. and Schmidhuber J. (2010), ‘Modeling attacks on physical unclonable functions’, Proc. 17th ACM Conf. on Computer and Communications Security (CCS), pp.237-249.

17. Schrijen G.-J. and van der Leest V. (2012), ‘Comparative analysis of SRAM memories used as PUF primitives’, Proc. Design, Automation and Test in Europe (DATE), pp.1319-1324.

18. Suh G.E. and Devadas S. (2007), ‘Physical unclonable functions for device authentication and secret key generation’, Proc. 44th Design Automation Conf. (DAC), pp.9-14.

19. Turan M.S., Barker E., Kelsey J., McKay K.A., Baish M.L. and Boyle M. (2018), ‘Recommendation for the entropy sources used for random bit generation’, NIST Special Publication 800-90B, National Institute of Standards and Technology.

20. Wilde F., Gammel B.M. and Pehl M. (2018), ‘Spatial correlation analysis on physical unclonable functions’, IEEE Transactions on Information Forensics and Security, Vol. 13, No. 6, pp.1468-1480.

21. Xiao K., Rahman M.T., Forte D., Huang Y., Su M. and Tehranipoor M. (2014), ‘Bit selection algorithm suitable for high-volume production of SRAM-PUF’, Proc. IEEE Int. Symp. on Hardware-Oriented Security and Trust (HOST), pp.101-106.